# Proyecto de software con Docker: API de tareas del equipo en dos contenedores comunicados

Entrega 1, Proyecto 1, Escenario 3

Leonardo Espinosa Rivera

Daniel Santiago Estupiñán Cruz

Anyelo Brayan Galindo Delgado

Jerlinson Ramos Galindez

Carlos Fernando Guerra Arango

[CONFIRMAR: en la portada solo van quienes trabajaron en la entrega; borrar los nombres que no apliquen]

Códigos estudiantiles: [CÓDIGO LEONARDO] [CÓDIGO DANIEL] [CÓDIGO ANYELO] [CÓDIGO JERLINSON]
Carlos Fernando Guerra Arango: 100360894

Programa de Ingeniería de Software

Facultad de Ingeniería, Diseño e Innovación

Politécnico Grancolombiano

TIC 42711: Integración Continua, grupo B04

Grupo de trabajo colaborativo 14

Tutor: Edward Reyes Corredor

Noviembre de 2026

**Tabla de contenido**

Introducción ..... 4

Objetivo general ..... 5

Objetivos específicos ..... 5

Desarrollo ..... 6

Repositorio en GitHub y estructura del proyecto ..... 6

Elección del proyecto y de las herramientas ..... 6

Estructura del repositorio ..... 7

Control de versiones y publicación en GitHub ..... 9

Arquitectura de los dos contenedores ..... 12

El Dockerfile explicado ..... 13

El archivo docker-compose.yml explicado ..... 15

Comunicación entre los contenedores ..... 18

Red y resolución de nombres ..... 19

La API consulta la base de datos ..... 20

Comprobación cruzada contra la base de datos ..... 23

Persistencia de los datos ..... 26

Pruebas automatizadas ..... 27

Conexión entre GitHub y Docker ..... 28

Cómo reproducir el proyecto ..... 30

Conclusiones ..... 31

Referencias ..... 32

## Introducción

La integración continua es una práctica del desarrollo de software en la que los integrantes de un equipo integran su trabajo con frecuencia y cada integración se verifica de forma automática, de modo que los errores se detecten pronto (Reyes Corredor, 2026). Para que esa práctica sea posible, el proyecto debe poder construirse y ejecutarse de la misma manera en cualquier equipo. Docker resuelve ese punto: empaqueta la aplicación y sus dependencias en contenedores que se comportan igual en el computador de cada integrante y, más adelante, en un servidor de integración continua.

A diferencia de una máquina virtual, que instala un sistema operativo completo sobre un hipervisor, Docker aprovecha el núcleo del sistema operativo del equipo anfitrión y empaqueta solo la aplicación con sus librerías, su configuración y sus dependencias, lo que permite que funcione igual en cualquier computador (Reyes Corredor, 2026).

Este documento corresponde a la Entrega 1 del proyecto del módulo. El enunciado indica que el proyecto se crea en un repositorio de GitHub y que, con Docker, se construyen dos contenedores comunicados entre sí. Para cumplirlo se desarrolló una API REST de registro de tareas del equipo con Node.js y Express, que guarda sus datos en una base de datos PostgreSQL. La API corre en un contenedor y la base de datos en otro; ambos se orquestan con Docker Compose. El dominio es deliberadamente sencillo, porque las entregas siguientes (Jenkins, Travis CI y Codeship) necesitan una aplicación con pruebas automatizadas sobre la cual construir los flujos de integración continua. El repositorio del proyecto está disponible en https://github.com/Owito/poli-ic-proyecto1.

Cada captura de pantalla que aparece en el documento es la salida real de un comando ejecutado durante la elaboración del trabajo, y va acompañada de una explicación de lo que muestra y de por qué importa para el proyecto.

**Objetivo general**

Construir y ejecutar con Docker una aplicación de dos contenedores, una API en Node.js y una base de datos PostgreSQL, que se comuniquen entre sí, y documentar con evidencias el proceso para que sirva de base a las entregas de integración continua.

**Objetivos específicos**

Primero, organizar el proyecto en un repositorio de control de versiones con una estructura clara, publicarlo en GitHub y compartirlo con los integrantes del grupo y con el tutor.

Segundo, definir un Dockerfile para la API y un archivo de Docker Compose que levante los dos servicios en una red propia, con volumen persistente para la base de datos, comprobación de salud y orden de arranque.

Tercero, demostrar con comandos reales que los dos contenedores se comunican: que la API consulta la base de datos y que los datos devueltos provienen de ella.

Cuarto, escribir pruebas automatizadas que puedan ejecutarse más adelante desde las herramientas de integración continua.

# Desarrollo

## Repositorio en GitHub y estructura del proyecto

### *Elección del proyecto y de las herramientas*

El tutor indicó que cada grupo escoge libremente el proyecto, siempre que sea proporcional y permita evidenciar que las herramientas del módulo se integran, y que se prefieren herramientas libres, instalando solo las necesarias para el proyecto (Reyes Corredor, 2026). Por eso se eligió un registro de tareas del equipo: tiene un modelo de datos de una sola tabla, un CRUD completo y suficiente lógica de validación para escribir pruebas útiles. Todas las herramientas usadas son libres o de uso gratuito: Git para el control de versiones, GitHub como servicio del repositorio remoto, Docker y Docker Compose para los contenedores, Node.js 22 con el marco Express para la API (Express, s. f.) y PostgreSQL 16 como base de datos (PostgreSQL Global Development Group, s. f.). Node.js y Git se instalan en el equipo porque se usan para escribir el código y las pruebas; PostgreSQL no se instala en el equipo, solo corre dentro de su contenedor. Como indicó el tutor, no se documenta el paso detallado de la instalación de Docker y de Git; la Figura 1 deja constancia de que están instalados y de las versiones con las que se trabajó (Reyes Corredor, 2026).

**Figura 1**
*Versiones de las herramientas utilizadas*



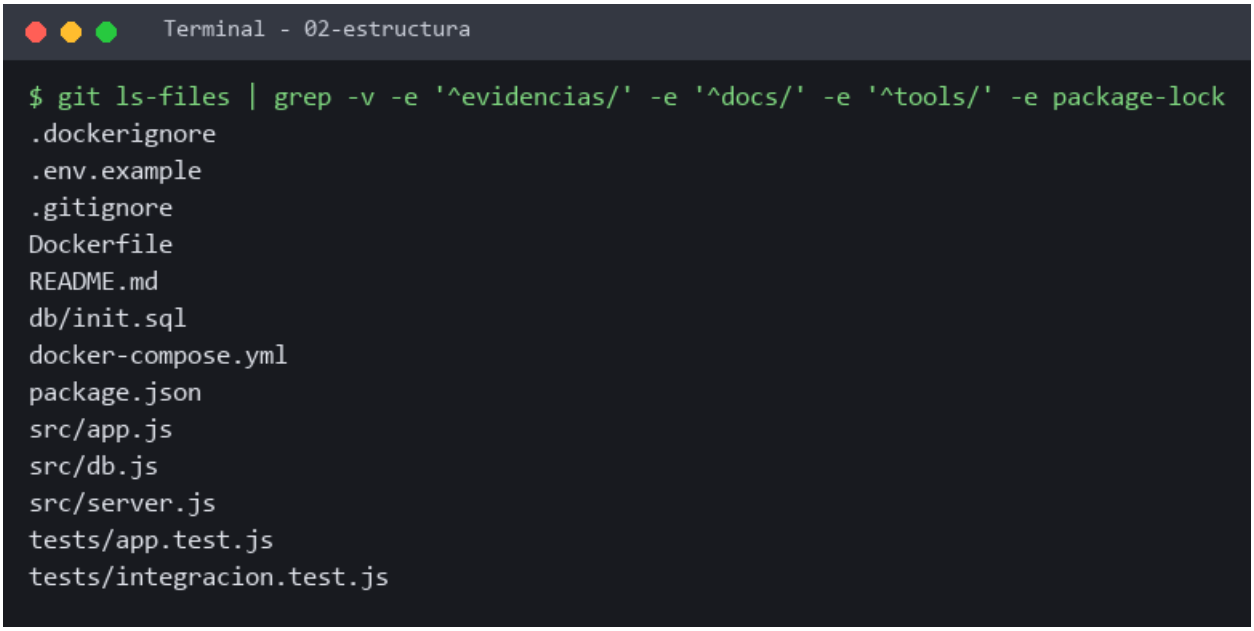

*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 1.** El comando imprime las versiones de Docker, Docker Compose, Git y Node.js instaladas en el equipo donde se construyó el proyecto. Sirve para que cualquier persona que reproduzca el trabajo sepa con qué versiones se verificó, y se relaciona con la recomendación del tutor de que todos trabajen con la misma versión de las herramientas.

***Estructura del repositorio***

El código se organiza de manera que cada archivo tenga una sola responsabilidad. La Figura 2 lista los archivos versionados con Git, y la Tabla 1 explica para qué sirve cada uno.

**Figura 2**
*Archivos versionados en el repositorio*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 2.** Es la salida de git ls-files, es decir, solo los archivos que Git está siguiendo. Se observa que el archivo .env con los valores reales no aparece en la lista, mientras que .env.example sí: las claves de la base de datos no se publican, solo su plantilla.

**Tabla 1**
*Archivos principales del proyecto*

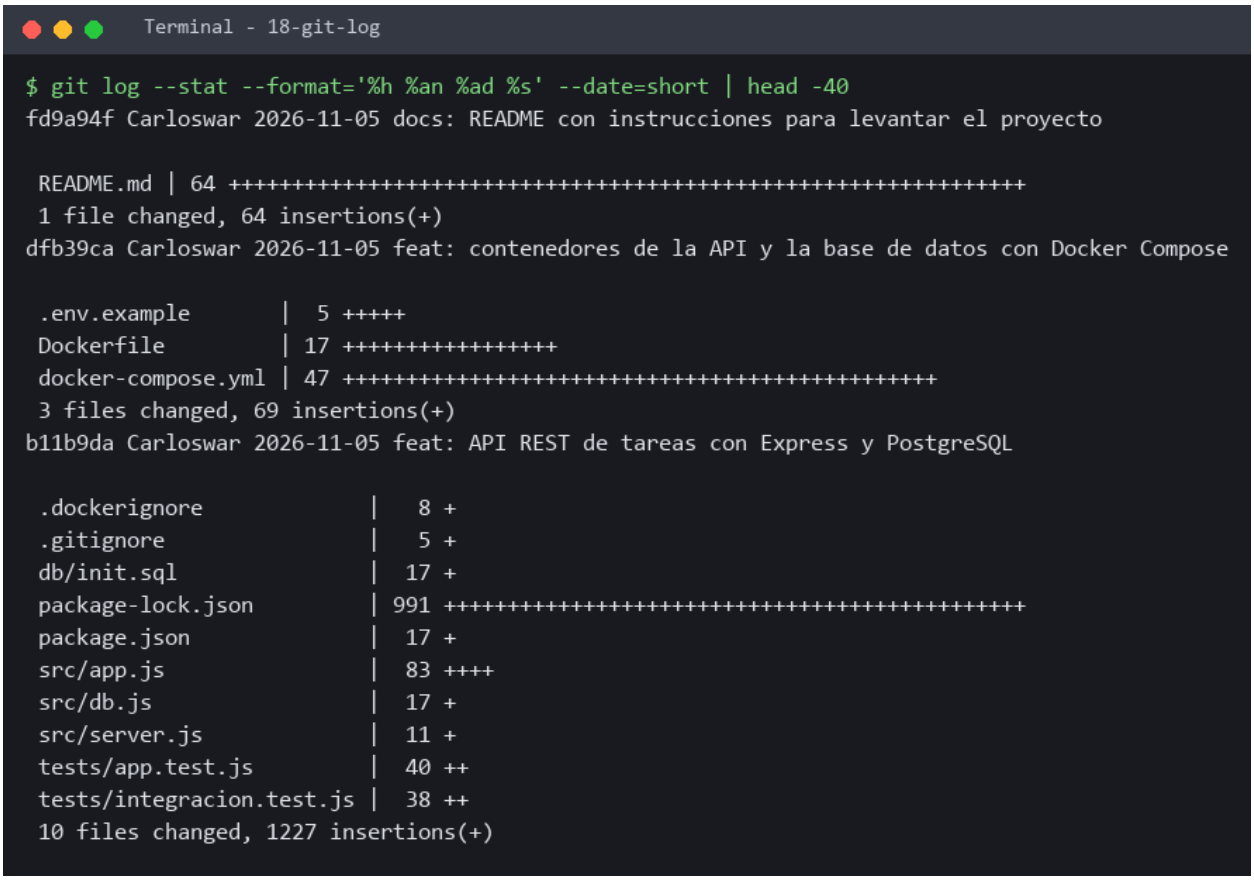
| Archivo | Función |
|---|---|
| Dockerfile | Receta para construir la imagen de la API en dos etapas, con usuario sin privilegios. |

| | |
|---|---|
| docker-compose.yml | Define los servicios app y db, la red red-tareas, el volumen tareas-datos, el healthcheck y el orden de arranque. |
| .env.example | Plantilla de variables de entorno sin secretos reales; cada integrante la copia a .env. |
| db/init.sql | Crea la tabla tareas y carga cinco filas de ejemplo la primera vez que se crea el volumen. |
| src/app.js, db.js, server.js | Rutas de la API, conexión a PostgreSQL y arranque del servidor. |
| tests/ | Pruebas unitarias (base de datos simulada) y de integración (base de datos real). |
| .dockerignore, .gitignore | Evitan que node_modules, .env y las evidencias entren a la imagen o al repositorio. |
| README.md | Instrucciones en español para levantar el proyecto. |

### *Control de versiones y publicación en GitHub*

El proyecto se desarrolló en un repositorio Git con commits pequeños y mensajes descriptivos que siguen la convención de tipos (feat, docs, test). La Figura 3 muestra el historial local con los archivos que cambió cada commit, un tipo de evidencia que el tutor mencionó como valiosa para ver el avance del equipo (Reyes Corredor, 2026).

**Figura 3**
*Historial de commits del repositorio*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 3.** Cada línea de cabecera es un commit con su identificador corto, autor, fecha y mensaje, seguida de los archivos que modificó. Confirma que el trabajo quedó registrado por etapas: primero la API, luego los contenedores y después la documentación.

El repositorio remoto se creó el 5 de noviembre de 2026 con la herramienta de línea de comandos de GitHub, gh, y después se enlazó con el repositorio local para subir el historial. La

Tabla 2 resume ese paso a paso. El mismo resultado se obtiene desde la página web de GitHub, con la opción New repository del menú que abre el signo más, arriba a la derecha, como lo mostró el tutor en el encuentro del 5 de noviembre (Reyes Corredor, 2026).

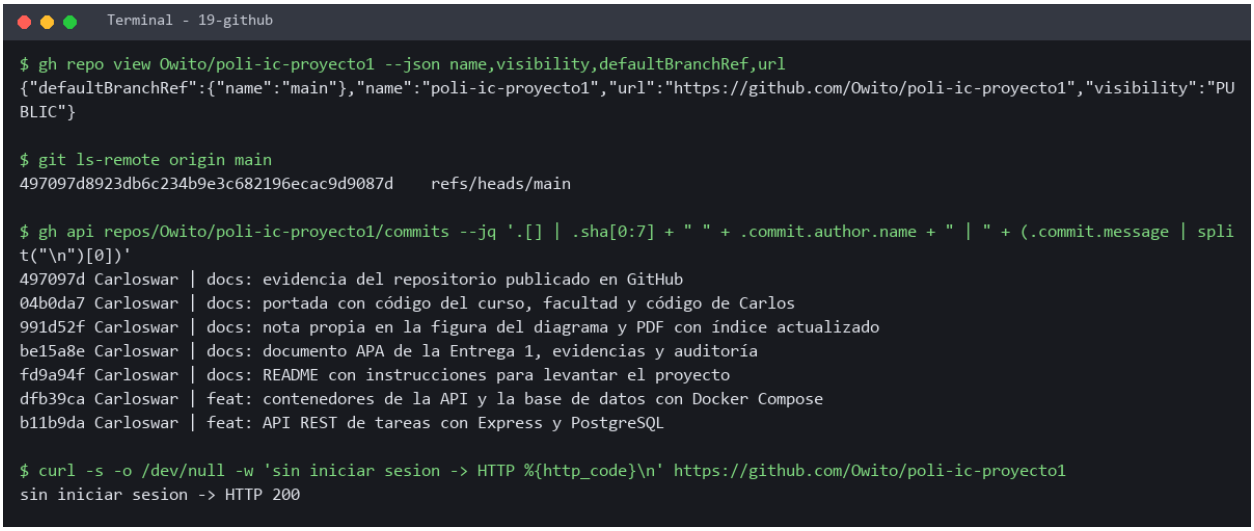
**Tabla 2**

*Pasos para crear el repositorio en GitHub y publicar el proyecto*

| Paso | Comando | Qué hace |
|---|---|---|
| 1 | gh repo create Owito/poli-ic-proyecto1 --private --description "Proyecto de Integración Continua: API de tareas en Node.js y PostgreSQL con Docker Compose" | Crea el repositorio vacío en la cuenta de GitHub, con su descripción. |
| 2 | git remote add origin https://github.com/Owito/poli-ic-proyecto1.git | Registra el repositorio de GitHub como remoto origin del repositorio local. |
| 3 | git push -u origin HEAD:main | Sube los commits a la rama main de GitHub y deja configurado el seguimiento entre la rama local y la remota. |
| 4 | gh repo edit Owito/poli-ic-proyecto1 --visibility public --accept-visibility-change-consequences | Cambia la visibilidad a pública, como recomendó el tutor, para que pueda entrar con el enlace. |

Los repositorios de GitHub se describen en su documentación oficial (GitHub, s. f.-b). El proyecto está publicado en un repositorio público de GitHub, https://github.com/Owito/poli-ic-proyecto1, enlazado como remoto origin del repositorio local. Se creó como privado y se cambió a público por recomendación del tutor, para que él pueda entrar a validarlo y para que las herramientas de integración continua muestren los errores donde los encuentren (Reyes Corredor, 2026). La Figura 4 muestra la consulta del repositorio publicado y su historial en GitHub.

**Figura 4**
*Repositorio publicado en GitHub*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 4.** La primera consulta confirma el nombre, la visibilidad pública, la rama principal main y la dirección del repositorio. La segunda muestra que la rama main de GitHub apunta al mismo commit que el repositorio local, y la tercera lista el historial tal como quedó publicado. La cuarta pide la página del repositorio sin iniciar sesión y recibe la respuesta HTTP 200: cualquier persona con el enlace, incluido el tutor, puede abrirlo.

Para revisar el proyecto, el tutor pidió entrar al repositorio y ver cómo están conformados el repositorio y los contenedores. En el encuentro del 5 de noviembre de 2026 aclaró que no

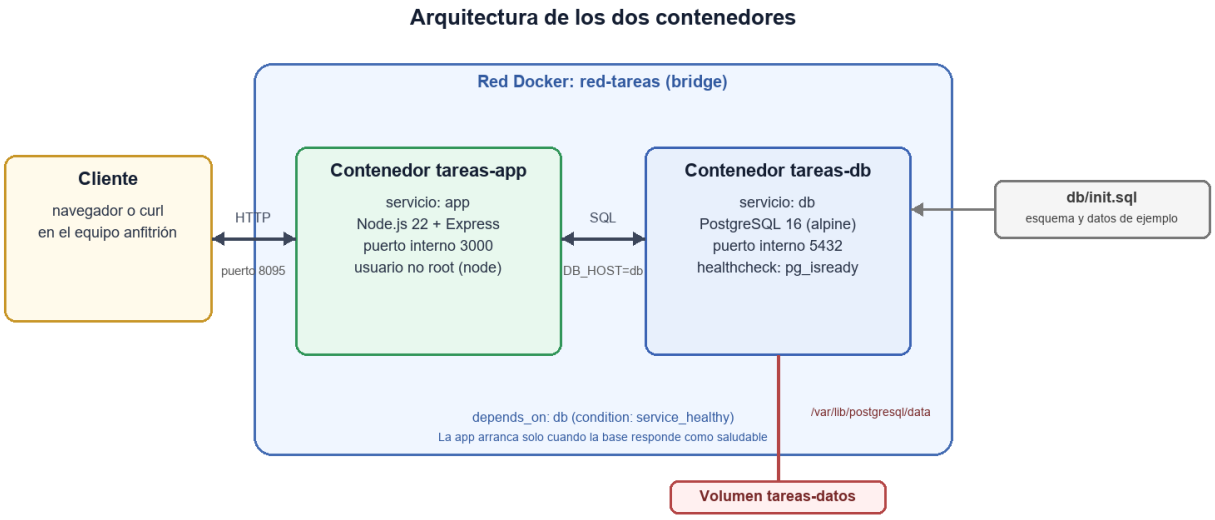
comparte su usuario de GitHub y que acepta tres alternativas: crearle un usuario, dejar el repositorio público o enviarle un enlace que le permita ingresar (Reyes Corredor, 2026). Con el repositorio público, el enlace anterior basta para que el tutor lo consulte sin cuenta ni invitación. Para que cada integrante pueda subir sus propios cambios, se le registra como colaborador en Settings, Collaborators con su usuario de GitHub, la opción que el tutor indicó para conformar el equipo de desarrollo y que GitHub documenta en su procedimiento de invitación (GitHub, s. f.-a).

**Arquitectura de los dos contenedores**

La solución tiene dos servicios que Docker Compose crea y conecta en una misma red. El primero, app, ejecuta la API de Node.js. El segundo, db, ejecuta PostgreSQL 16. Solo la API se publica hacia el equipo anfitrión, por el puerto 8095; la base de datos queda accesible únicamente dentro de la red, lo que reduce la superficie expuesta.

**Figura 5**

*Diagrama de la arquitectura de la solución*




*Nota.* Elaboración propia.

**Qué muestra la Figura 5.** El cliente (navegador o curl) habla por HTTP con el contenedor tareas-app, que atiende en el puerto interno 3000 y está publicado en el 8095 del

equipo. Ese contenedor se conecta por SQL con tareas-db usando el nombre de servicio db como dirección. Ambos viven en la red red-tareas. La base de datos guarda sus archivos en el volumen tareas-datos y recibe el script db/init.sql en el primer arranque. La flecha de depends_on indica que la API no arranca hasta que la base de datos pase su comprobación de salud.

**Tabla 3**
*Los dos contenedores del proyecto*

| Servicio | Contenedor | Imagen | Puerto | Responsabilidad |
|---|---|---|---|---|
| app | tareas-app | Construida desde el Dockerfile (Node.js 22 sobre Alpine) | 3000 interno, 8095 publicado | API REST de tareas y endpoint /salud |
| db | tareas-db | postgres:16-alpine | 5432 solo dentro de la red | Almacena las tareas de forma persistente |

## El Dockerfile explicado

El Dockerfile es la receta con la que Docker construye la imagen de la API (Docker Inc., s. f.-c). Se escribió en dos etapas, como muestra el Listado 1.

**Listado 1**
*Dockerfile de la API*

```
# Etapa 1: instala solo las dependencias de producción.
FROM node:22-alpine AS dependencias
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

# Etapa 2: imagen final, mínima y sin privilegios de root.
FROM node:22-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY --from=dependencias /app/node_modules ./node_modules
COPY package.json ./
COPY src ./src
COPY tests ./tests
USER node
EXPOSE 3000
CMD ["node", "src/server.js"]
```

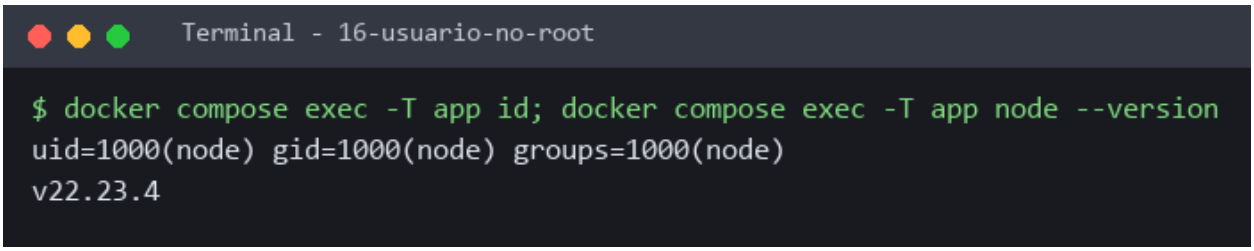
*Nota.* Contenido del archivo Dockerfile del repositorio.

En la primera etapa, llamada dependencias, se copian solo package.json y package-lock.json y se ejecuta npm ci --omit=dev. Así se instalan exactamente las versiones bloqueadas y únicamente las dependencias de producción. Como Docker guarda cada instrucción en una capa,

mientras esos dos archivos no cambien, la instalación se reutiliza de la memoria de construcción y las reconstrucciones son rápidas.

La segunda etapa parte de una imagen limpia de node:22-alpine y copia desde la primera únicamente la carpeta node_modules, junto con el código de src y tests. El resultado es una imagen final pequeña, sin herramientas de construcción. La instrucción USER node hace que el proceso corra con un usuario sin privilegios, en lugar de root, una práctica de seguridad que limita el daño si la aplicación llegara a ser vulnerada. La Figura 6 lo comprueba dentro del contenedor en ejecución.

**Figura 6**
*Usuario con el que corre la API dentro del contenedor*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 6.** El comando id, ejecutado dentro del contenedor tareas-app, devuelve uid=1000(node): el proceso no es root. El segundo comando confirma que la versión de Node.js del contenedor es la 22, la que pide el proyecto.

El archivo .dockerignore evita que node_modules, el repositorio .git, el archivo .env y las evidencias se envíen al motor de Docker al construir. Con ello el contexto de construcción es pequeño y los secretos locales no terminan dentro de la imagen.

**El archivo docker-compose.yml explicado**

Docker Compose permite describir varios contenedores y sus relaciones en un solo archivo (Docker Inc., s. f.-a; Docker Inc., s. f.-b). El Listado 2 contiene el archivo del proyecto.

**Listado 2**

*Archivo docker-compose.yml*

```
name: tareas-equipo

services:
  db:
    image: postgres:16-alpine
    container_name: tareas-db
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - tareas-datos:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
    networks:
      - red-tareas
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped

  app:
    build: .
    container_name: tareas-app
    environment:
      PORT: 3000
      DB_HOST: db
      DB_PORT: 5432
      DB_USER: ${POSTGRES_USER}
      DB_PASSWORD: ${POSTGRES_PASSWORD}
      DB_NAME: ${POSTGRES_DB}
    ports:
      - "${APP_PUERTO:-8095}:3000"
    networks:
      - red-tareas
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

networks:
  red-tareas:
    driver: bridge

volumes:
  tareas-datos:
```

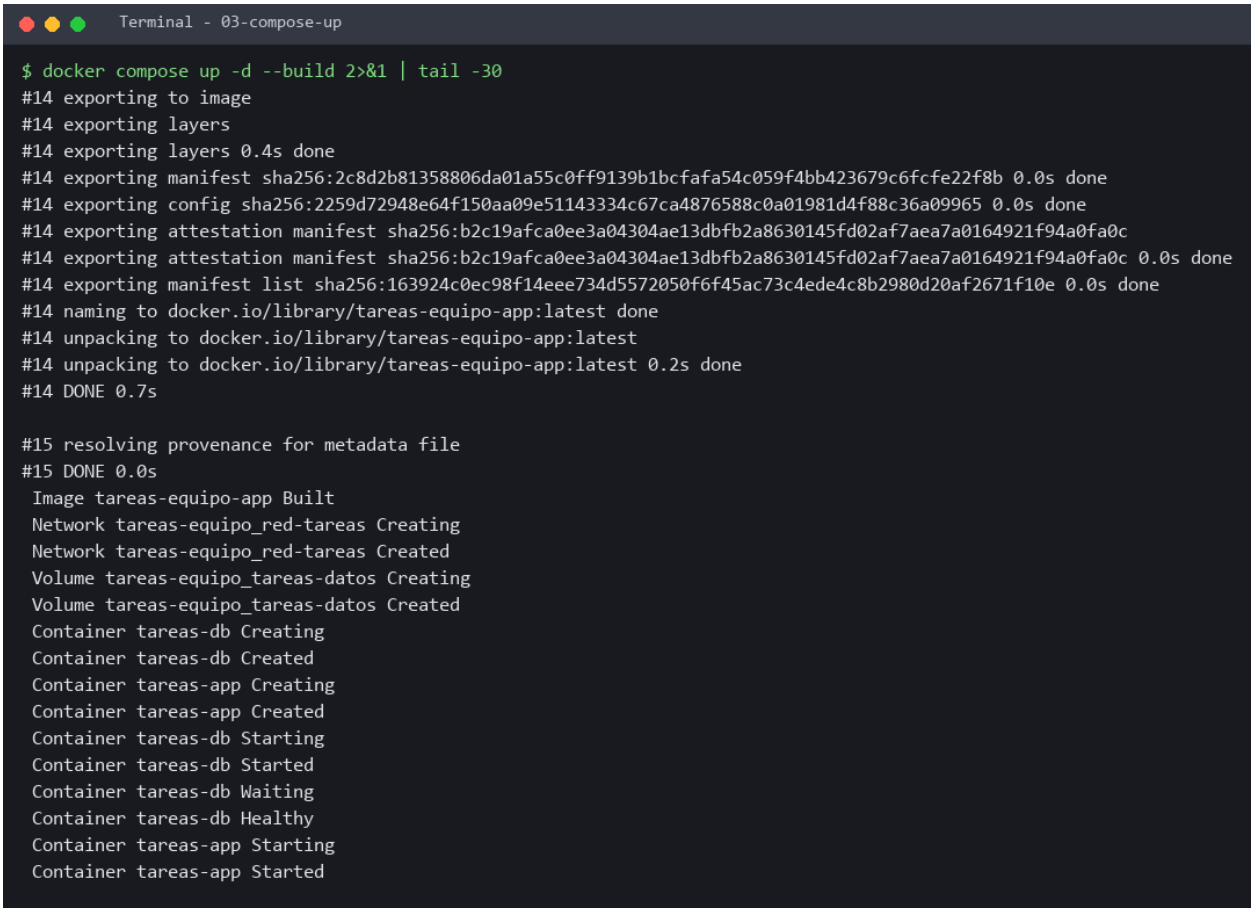
*Nota.* Contenido del archivo docker-compose.yml del repositorio.

Los puntos clave son los siguientes. **Red definida:** ambos servicios se conectan a red-tareas, una red de tipo bridge creada por el propio archivo. En una red definida por el usuario, Docker resuelve el nombre de cada servicio como dirección, por eso la API usa DB_HOST=db (Docker Inc., s. f.-d). **Volumen:** tareas-datos se monta en /var/lib/postgresql/data, de modo que las tareas sobreviven aunque se borren los contenedores (Docker Inc., s. f.-f). **Script de inicialización:** el archivo db/init.sql se monta en la carpeta docker-entrypoint-initdb.d, y la imagen oficial de PostgreSQL lo ejecuta una sola vez, cuando el volumen está vacío (Docker Inc., s. f.-e). **Healthcheck:** la base de datos se declara saludable cuando pg_isready responde, con una prueba cada cinco segundos. **Orden de arranque:** depends_on con condition: service_healthy hace que la API espere a que la base de datos esté lista, evitando errores de conexión al inicio. **Variables de entorno:** el usuario, la clave y el nombre de la base se leen del archivo .env, que no se sube al repositorio; el repositorio solo contiene .env.example.

La Figura 7 muestra el arranque real. Docker construye la imagen, crea la red y el volumen, inicia primero la base de datos, espera a que sea saludable y solo entonces inicia la API.

**Figura 7**

*Construcción y arranque con docker compose up*

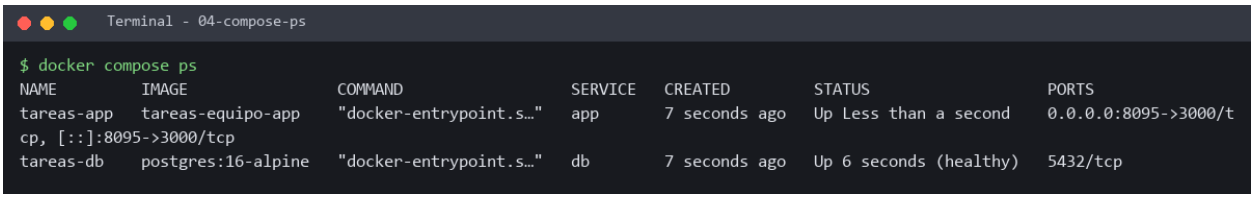



*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 7.** Se ven los últimos pasos de la construcción de la imagen tareas-equipo-app y, a continuación, la creación de la red tareas-equipo_red-tareas, del volumen tareas-equipo_tareas-datos y de los dos contenedores. El orden de las líneas es la prueba de la dependencia: tareas-db pasa por Waiting y luego Healthy, y recién después tareas-app pasa a Starting y Started.

**Figura 8**

*Estado de los dos contenedores*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 8.** docker compose ps lista los dos servicios del proyecto. tareas-db aparece con estado healthy, lo que significa que su healthcheck está pasando, y tareas-app aparece en ejecución con el puerto 8095 del equipo redirigido al 3000 del contenedor. La base de datos solo muestra su puerto interno 5432, sin publicarlo hacia afuera.

La Figura 8 también permite distinguir imagen y contenedor. La imagen es la plantilla a partir de la cual se crea el contenedor; el tutor la comparó con el plano de una casa, y al contenedor con la casa ya construida (Reyes Corredor, 2026). En este proyecto hay dos imágenes, tareas-equipo-app, construida desde el Dockerfile, y postgres:16-alpine, descargada de Docker Hub, y de cada una se crea un contenedor: tareas-app y tareas-db.

**Figura 9**

*Estado de salud reportado por Docker*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 9.** docker inspect lee directamente el estado de salud que Docker registra para tareas-db: running y healthy. Es la misma información que usa depends_on para decidir cuándo arrancar la API.

**Comunicación entre los contenedores**

Que los dos contenedores estén en ejecución no prueba por sí solo que se comuniquen. Por eso se verificó la comunicación desde cuatro ángulos: la red de Docker, la resolución de

nombres, una respuesta de la API que contiene datos de la base y la comprobación cruzada directamente contra la base de datos.

***Red y resolución de nombres***

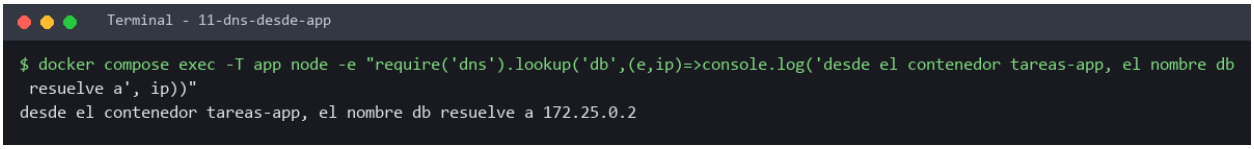
**Figura 10**
*Contenedores conectados a la red red-tareas*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 10.** docker network inspect lista los miembros de la red tareas-equipo_red-tareas, de tipo bridge. Aparecen tareas-app y tareas-db, cada uno con su dirección IP dentro de la misma subred. Estar en la misma red es la condición para que puedan hablar entre sí.

**Figura 11**
*Resolución del nombre db desde el contenedor de la API*

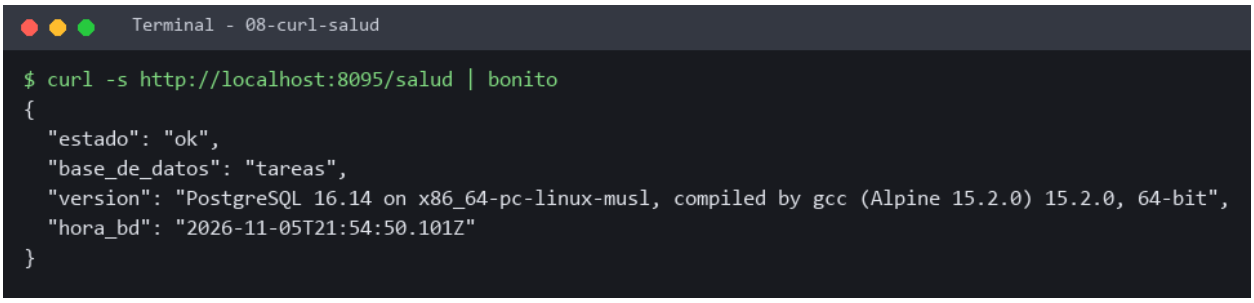



*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 11.** Se ejecuta un pequeño programa de Node.js dentro de tareas-app que pregunta por la dirección del nombre db. La respuesta coincide con la IP que la Figura 10 asigna a tareas-db. Es la prueba de que la API no necesita conocer direcciones numéricas: le basta con el nombre del servicio.

***La API consulta la base de datos***

**Figura 12**
*Respuesta del endpoint /salud*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 12.** El endpoint /salud ejecuta una consulta en la base de datos y devuelve su nombre (tareas) y la versión de PostgreSQL (16.x). Esa información no existe en la API: solo puede provenir del otro contenedor. Si la base dejara de responder, el endpoint devolvería HTTP 503 con estado error, comportamiento cubierto por una de las pruebas automatizadas.

**Figura 13**
*Listado de tareas devuelto por la API*

Terminal - 09-curl-tareas

```
$ curl -s http://localhost:8095/tareas | bonito
[
  {
    "id": 1,
    "titulo": "Crear el repositorio del proyecto en GitHub",
    "responsable": "Integrante 1",
    "estado": "hecha",
    "creada_en": "2026-11-05T21:54:45.154Z"
  },
  {
    "id": 2,
    "titulo": "Escribir el Dockerfile de la API",
    "responsable": "Integrante 2",
    "estado": "hecha",
    "creada_en": "2026-11-05T21:54:45.154Z"
  },
  {
    "id": 3,
    "titulo": "Definir docker-compose con la API y la base de datos",
    "responsable": "Integrante 3",
    "estado": "en_progreso",
    "creada_en": "2026-11-05T21:54:45.154Z"
  },
  {
    "id": 4,
    "titulo": "Redactar las pruebas automatizadas",
    "responsable": "Integrante 4",
    "estado": "pendiente",
    "creada_en": "2026-11-05T21:54:45.154Z"
  },
  {
    "id": 5,
    "titulo": "Preparar el documento de la Entrega 1",
    "responsable": "Integrante 5",
    "estado": "pendiente",
    "creada_en": "2026-11-05T21:54:45.154Z"
  }
]
```

*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 13.** La API devuelve en formato JSON las cinco tareas de ejemplo que el script db/init.sql cargó en la base de datos al crearse el volumen. Los datos del navegador o de curl viajan por HTTP hasta tareas-app y esta los obtiene de tareas-db con una consulta SQL.

***Comprobación cruzada contra la base de datos***

**Figura 14**

*Consulta directa a PostgreSQL con psql*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 14.** Con docker compose exec se abre psql dentro del contenedor tareas-db y se consulta la tabla tareas. Las cinco filas coinciden con las que entregó la API en la Figura 13, lo que confirma que la API lee de esta base de datos y no de datos escritos en su propio código.

**Figura 15**

*Creación y actualización de una tarea por la API*

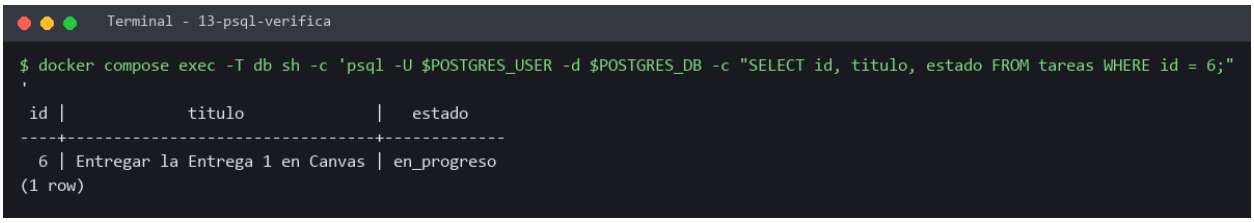



*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 15.** Con curl se envía un POST que crea la tarea 6 y un PUT que cambia su estado a en_progreso. La API responde con el registro completo, incluida la fecha creada_en que asigna la base de datos.

**Figura 16**

*La tarea creada por la API existe en la base de datos*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 16.** Una consulta con psql a la fila 6 devuelve la tarea creada en la Figura 15 con su estado actualizado. Es la comprobación de ida y vuelta: lo que la API escribe queda guardado en el contenedor de la base de datos.

**Figura 17**

*Respuestas de error de la API*



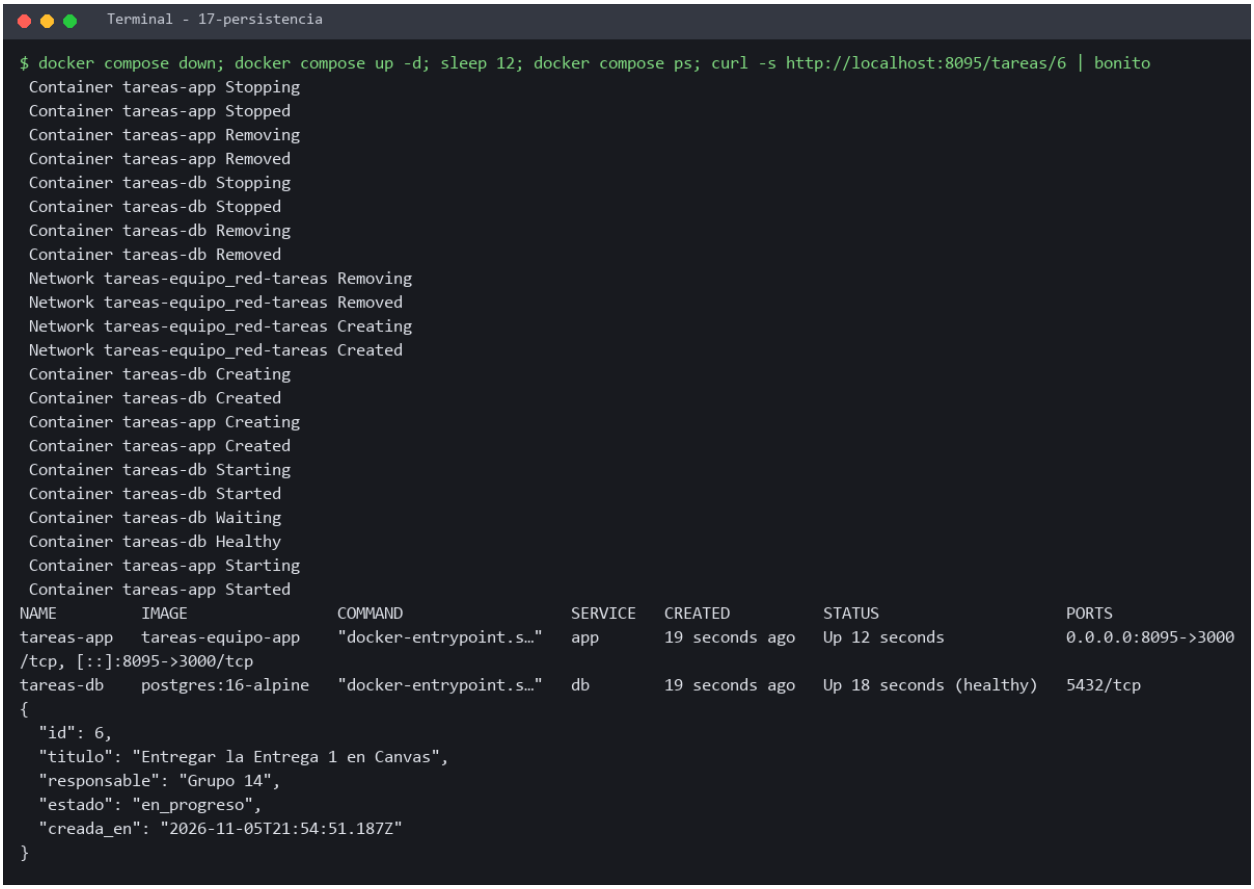

*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 17.** Un POST sin título responde HTTP 400 y la consulta de una tarea inexistente responde HTTP 404. La validación ocurre en la API antes de tocar la base, y las rutas inexistentes se manejan con códigos HTTP correctos.

### *Persistencia de los datos*

El volumen tareas-datos guarda los datos fuera del ciclo de vida de los contenedores. Para comprobarlo se ejecutó docker compose down, que borra los contenedores y la red pero no el volumen, y luego docker compose up -d.

**Figura 18**
*Los datos sobreviven a la recreación de los contenedores*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

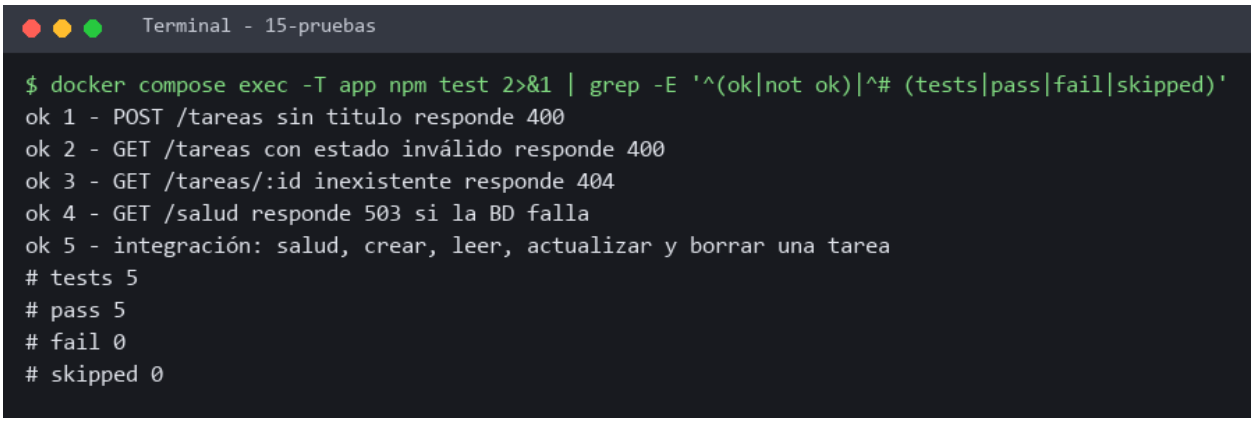
**Qué muestra la Figura 18.** Se ven la eliminación y la nueva creación de los dos contenedores y de la red. Después de que tareas-db vuelve a estar healthy, la consulta a la tarea 6, creada antes del reinicio, sigue respondiendo. Según la documentación de la imagen oficial, el script de inicialización solo se ejecuta cuando el volumen está vacío, por lo que no se vuelve a ejecutar en este reinicio (Docker Inc., s. f.-e).

**Pruebas automatizadas**

Las pruebas usan el ejecutor integrado de Node.js, node:test, que no requiere instalar dependencias adicionales (Node.js, s. f.). Hay dos grupos. Las pruebas unitarias reemplazan la base de datos por un objeto simulado y verifican la validación de entradas (HTTP 400), los casos de recurso inexistente (HTTP 404) y el comportamiento de /salud cuando la base falla (HTTP 503). La prueba de integración usa la base de datos real y recorre el ciclo completo: salud, listado de las tareas de ejemplo, creación, actualización, borrado y verificación del borrado. La prueba de integración se omite automáticamente cuando no existe la variable DB_HOST, de modo que el comando npm test también funciona en un equipo sin Docker.

**Figura 19**

*Ejecución de las pruebas dentro del contenedor de la API*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 19.** Dentro de tareas-app, donde la variable DB_HOST apunta a la base de datos, las cinco pruebas se ejecutan y pasan, sin pruebas omitidas, incluida la de integración contra el contenedor tareas-db. El resumen final reporta cinco pruebas, cinco aprobadas y cero fallidas.

Estas mismas pruebas se ejecutarán desde Jenkins en la Entrega 2 y desde Travis CI y Codeship en la Entrega 3, ya que el comando npm test no depende de ninguna herramienta específica.

**Conexión entre GitHub y Docker**

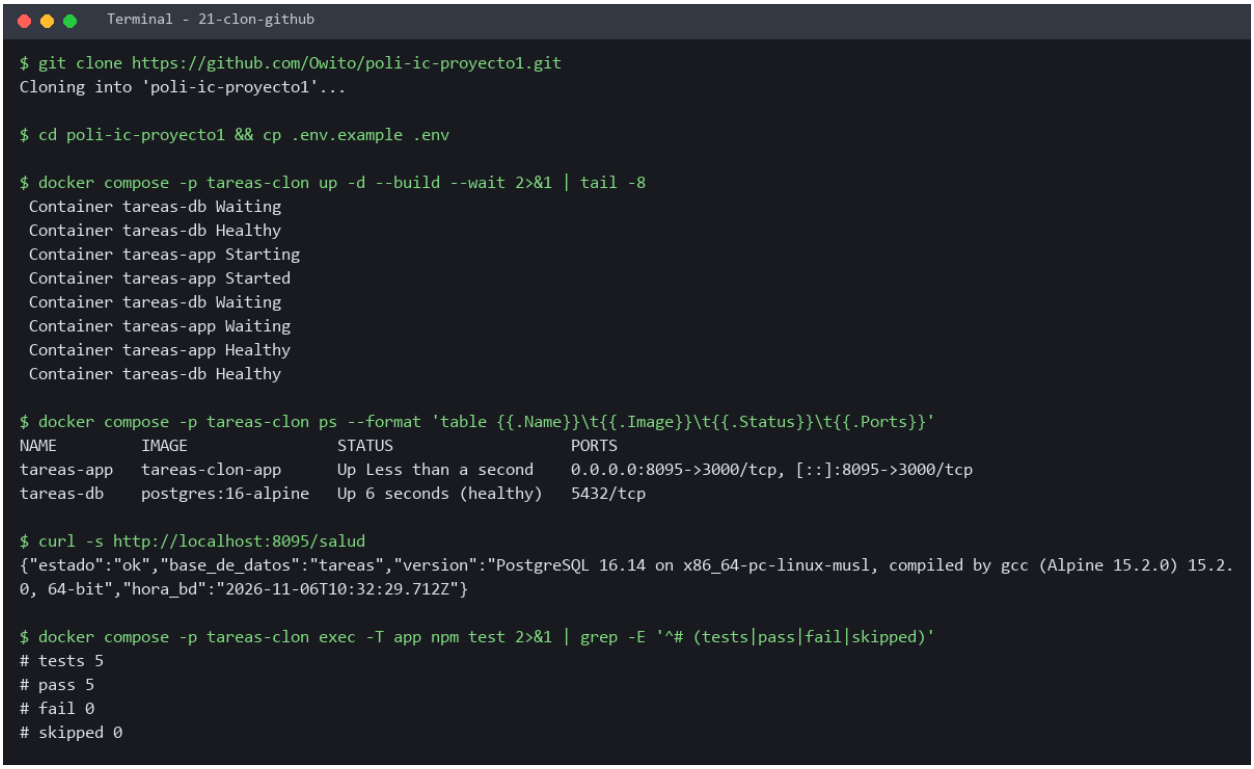
El tutor precisó que el alcance de esta entrega es trabajar con GitHub, construir los dos contenedores con Docker y conectar Docker con GitHub (Reyes Corredor, 2026). En este proyecto la conexión se da a través de los archivos que definen los contenedores. GitHub no guarda las imágenes ni los contenedores, sino las instrucciones para construirlos: el Dockerfile de la API, el archivo docker-compose.yml con los dos servicios, la red y el volumen, el script db/init.sql y la plantilla .env.example, todos visibles en la Figura 2. A partir de esos archivos, Docker construye la imagen de la API en el equipo de quien ejecuta el proyecto y descarga de Docker Hub la imagen oficial postgres:16-alpine (Docker Inc., s. f.-e).

Por eso el repositorio es el punto de encuentro del equipo: cualquier integrante, o el tutor, clona el repositorio y ejecuta docker compose up -d --build para obtener los mismos dos contenedores, con la misma red y la misma configuración. Cuando alguien modifica el Dockerfile o el docker-compose.yml, el cambio queda en el historial de commits y los demás lo reciben al actualizar su copia, lo que evita que cada integrante trabaje con una versión distinta del ambiente. Es también la conexión que aprovecharán Jenkins en la Entrega 2 y Travis CI y Codeship en la Entrega 3, que tomarán el código del repositorio de GitHub para construir los contenedores y ejecutar las pruebas.

Para comprobarlo, el proyecto se clonó desde el repositorio público en una carpeta nueva, sin iniciar sesión en GitHub, y se levantó con otro nombre de proyecto de Docker Compose, tareas-clon, para no reutilizar el volumen del ambiente original. La Figura 20 muestra el resultado.

**Figura 20**

*El proyecto clonado desde GitHub levanta los mismos dos contenedores*




*Nota.* Captura de la salida real del comando, generada durante la elaboración de este trabajo.

**Qué muestra la Figura 20.** El comando git clone descarga el repositorio público; después se copia la plantilla .env.example como .env y se levanta el proyecto. Docker construye la imagen de la API a partir del Dockerfile del repositorio y descarga postgres:16-alpine: los dos contenedores quedan saludables, el endpoint /salud responde con la versión de PostgreSQL leída de la base de datos y las cinco pruebas automatizadas pasan dentro del contenedor de la API, sin pruebas omitidas. Es la evidencia de que lo que está en GitHub basta para reconstruir el ambiente completo.

**Cómo reproducir el proyecto**

El archivo README.md del repositorio lo explica en español. En resumen: se clona el repositorio con git clone https://github.com/Owito/poli-ic-proyecto1.git, se copia .env.example como .env y se cambia la clave, se ejecuta docker compose up -d --build, se verifica con docker compose ps y se consultan http://localhost:8095/salud y http://localhost:8095/tareas. Para detenerlo se usa docker compose down.

## Conclusiones

Se construyó una aplicación de dos contenedores, una API en Node.js y una base de datos PostgreSQL, que se ejecuta con un solo comando y que se comunica a través de una red definida por el proyecto. La comunicación se demostró con evidencia verificable: la red contiene a ambos contenedores, el nombre db se resuelve desde la API, y los datos que devuelve la API coinciden con los de la base de datos. Además, como el repositorio de GitHub guarda la definición de los contenedores y no las imágenes, cualquier integrante obtiene el mismo ambiente a partir de una copia del repositorio y un solo comando, que es la conexión entre GitHub y Docker en la que se apoyarán las entregas siguientes.

El uso de healthcheck y depends_on con condición evita un error común de arranque, en el que la aplicación intenta conectarse antes de que la base de datos esté lista. El volumen asegura que los datos persistan, y la separación de configuración en .env permite publicar el proyecto sin exponer claves. El Dockerfile multietapa con usuario sin privilegios produce una imagen más pequeña y más segura.

Las pruebas automatizadas quedaron listas para integrarse en las siguientes entregas. Como el comando npm test funciona dentro y fuera de Docker, Jenkins, Travis CI y Codeship podrán ejecutarlo sin cambios en la aplicación. Como aprendizaje del grupo, la integración continua depende de contar primero con un entorno reproducible; Docker aporta esa base.

Como limitaciones, el proyecto aún no tiene un flujo de integración continua, que corresponde a la Entrega 2, y las pruebas de integración dependen de que la base de datos esté disponible.

## Referencias


Docker Inc. (s. f.-a). *Compose file reference*. Docker Docs. Recuperado el 5 de noviembre de 2026, de https://docs.docker.com/reference/compose-file/

Docker Inc. (s. f.-b). *Docker Compose*. Docker Docs. Recuperado el 5 de noviembre de 2026, de https://docs.docker.com/compose/

Docker Inc. (s. f.-c). *Dockerfile reference*. Docker Docs. Recuperado el 5 de noviembre de 2026, de https://docs.docker.com/reference/dockerfile/

Docker Inc. (s. f.-d). *Networking overview*. Docker Docs. Recuperado el 5 de noviembre de 2026, de https://docs.docker.com/engine/network/

Docker Inc. (s. f.-e). *postgres: Official image*. Docker Hub. Recuperado el 5 de noviembre de 2026, de https://hub.docker.com/_/postgres

Docker Inc. (s. f.-f). *Volumes*. Docker Docs. Recuperado el 5 de noviembre de 2026, de https://docs.docker.com/engine/storage/volumes/

Express. (s. f.). *Express: Node.js web application framework*. Recuperado el 5 de noviembre de 2026, de https://expressjs.com/

GitHub. (s. f.-a). *Inviting collaborators to a personal repository*. GitHub Docs. Recuperado el 5 de noviembre de 2026, de https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-personal-account-on-github/managing-access-to-your-personal-repositories/inviting-collaborators-to-a-personal-repository

GitHub. (s. f.-b). *Repositories documentation*. GitHub Docs. Recuperado el 5 de noviembre de 2026, de https://docs.github.com/en/repositories

Node.js. (s. f.). *Test runner*. Node.js v22 documentation. Recuperado el 5 de noviembre de 2026, de https://nodejs.org/docs/latest-v22.x/api/test.html

PostgreSQL Global Development Group. (s. f.). *PostgreSQL 16 documentation*. Recuperado el 5 de noviembre de 2026, de https://www.postgresql.org/docs/16/

Reyes Corredor, E. (2026, 19 y 26 de octubre, y 5 de noviembre). *Encuentros sincrónicos de Énfasis Profesional I (Integración Continua)* [Grabaciones de clase]. Politécnico Grancolombiano.